

Documento de Arquitetura de Software (DAS)

Sistema de Rastreamento de Ônibus

João Oliveira Duarte, Lorhan Santos de Lima,
Raul Etcheverry Reis e Vanessa Manzke Otte

Universidade Federal do Pampa (Unipampa)
Engenharia de Software 2026/1

1 Introdução

Este documento define as diretrizes arquiteturais para o Sistema de Rastreamento de Ônibus. O objetivo é estabelecer uma fundação técnica sólida que garanta que os Requisitos Funcionais e Não Funcionais (especificados no Documento de Requisitos) sejam atendidos. A arquitetura foca em resiliência offline, baixo consumo de bateria para os emissores e comunicação rápida para os consumidores.

2 Características e Funcionamento do Sistema

O sistema opera sob o modelo de **Cliente-Servidor**, atuando com dois perfis distintos de clientes conectados a uma API centralizada:

- **App Motorista (Produtor):** Funciona como um rastreador em segundo plano. O motorista seleciona a linha e inicia a rota. O sistema captura as coordenadas GPS a cada 15 segundos e as envia ao servidor.
- **App Passageiro (Consumidor):** Funciona como um cliente sob demanda. O aplicativo não mantém conexões persistentes abertas; ele solicita ao servidor a posição dos veículos apenas quando o usuário abre o mapa da linha.
- **Mecanismo de Resiliência (Store and Forward):** Característica central do sistema. Se o ônibus entrar em uma zona de sombra (sem sinal 3G/4G), o aplicativo do motorista intercepta a falha, salva as coordenadas no banco de dados local do celular e, assim que a rede é restabelecida, sincroniza o lote completo (Batch) com o servidor.

3 Stack Tecnológica

A seleção de tecnologias priorizou ferramentas consolidadas, gratuitas e de amplo suporte comunitário, adequadas para a construção de um MVP (Produto Mínimo Viável) acadêmico robusto:

- **Frontend (Aplicativos Móveis):** Desenvolvimento Nativo Android utilizando Java.

- **Padrão de Interface: MVVM** (Model-View-ViewModel).
- **Persistência Local (Offline): SQLite** nativo do Android.
- **Cliente HTTP (Android): Retrofit**.
- **Backend (Servidor da API): Python** com o microframework **Flask**.
- **Formato de Comunicação: JSON** via protocolo HTTPS.

4 Visão da Macroarquitetura

A macroarquitetura define a topologia física do sistema. Os aplicativos Android operam na borda (dispositivos físicos), conectando-se assincronamente ao servidor central na nuvem, que detém a "fonte da verdade" no banco de dados.

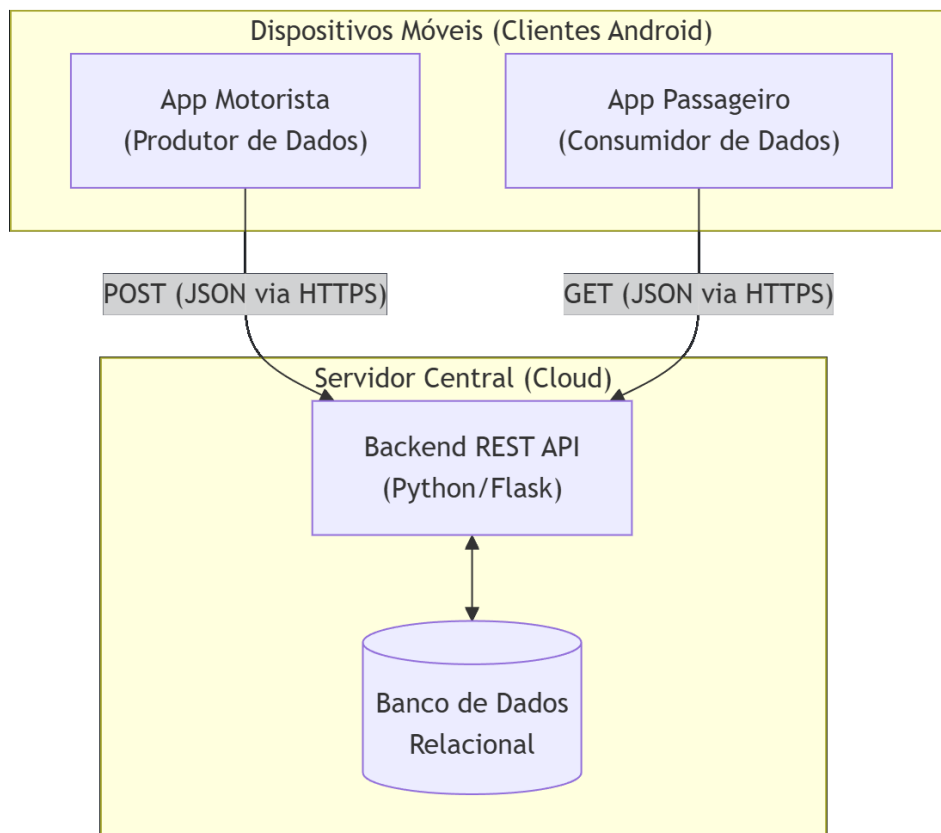


Figura 1: Diagrama de Implantação (Cliente-Servidor)

5 Visão da Microarquitetura - Frontend Android

O desenvolvimento dos aplicativos móveis seguirá o padrão **MVVM (Model-View-ViewModel)**. Esta escolha arquitetural separa a interface gráfica (View) das regras lógicas de apresentação (ViewModel) e do acesso aos dados externos e locais (Model).

Regras de negócio no frontend, como a defasagem temporal que define se o ícone do ônibus deve ser pintado de verde ou cinza, ocorrerão na camada ViewModel, desafogando o servidor.

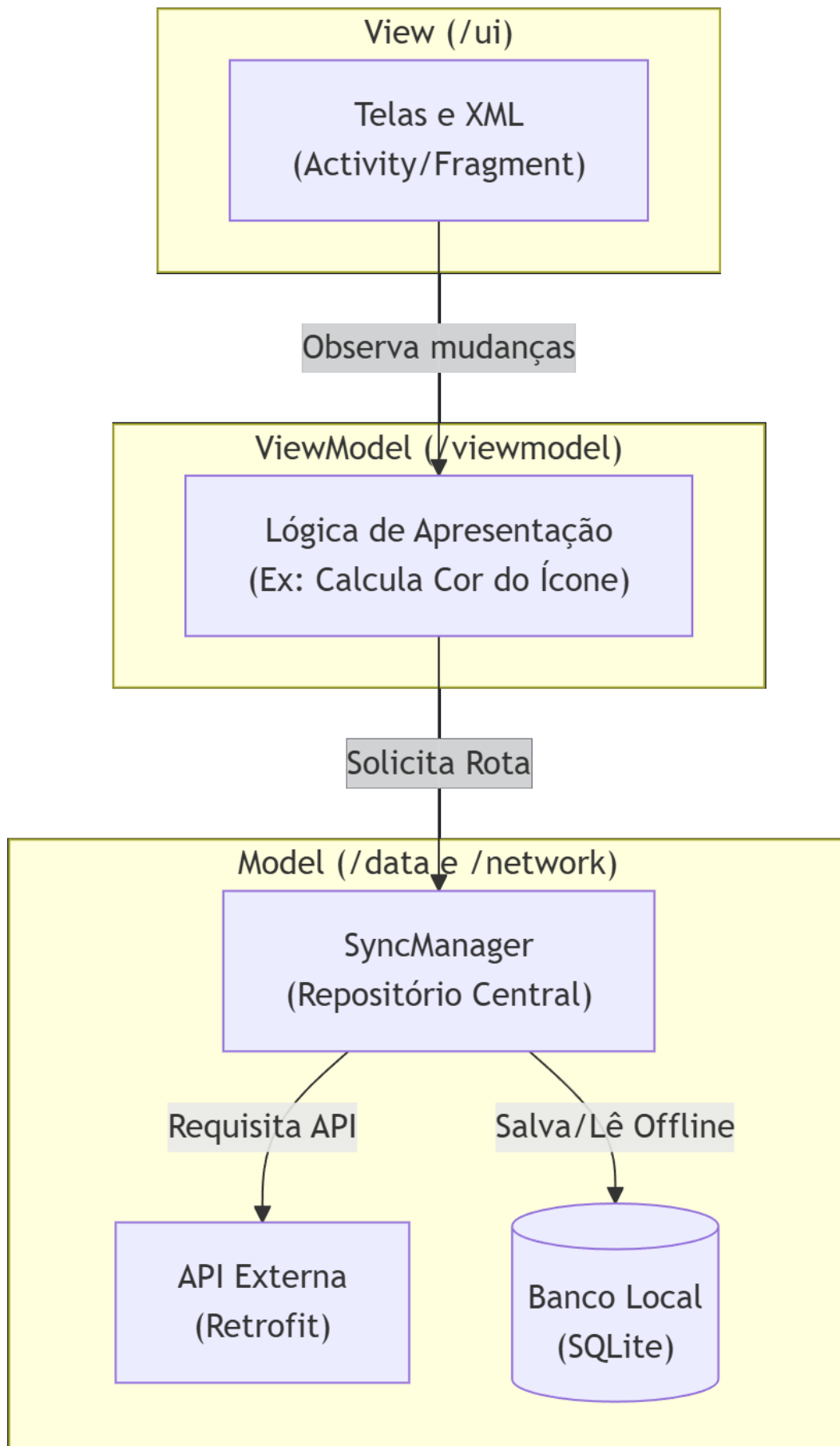


Figura 2: Diagrama de Componentes MVVM do Android

6 Visão da Microarquitetura - Backend Python

A API será desenvolvida em **Python/Flask** adotando uma arquitetura em camadas. O objetivo é evitar rotas monolíticas onde a validação, a regra de negócio e a consulta ao banco de dados residam no mesmo bloco de código.

- **Controllers (Rotas):** Recebem as requisições, validam se os cabeçalhos e o corpo da requisição estão no formato exigido pelo Contrato da API.
- **Services (Serviços):** Aplicam regras de negócio, como a moderação algorítmica de avisos da comunidade.
- **Repository (Acesso a Dados):** Isolam o código SQL, padronizando as operações de CRUD.

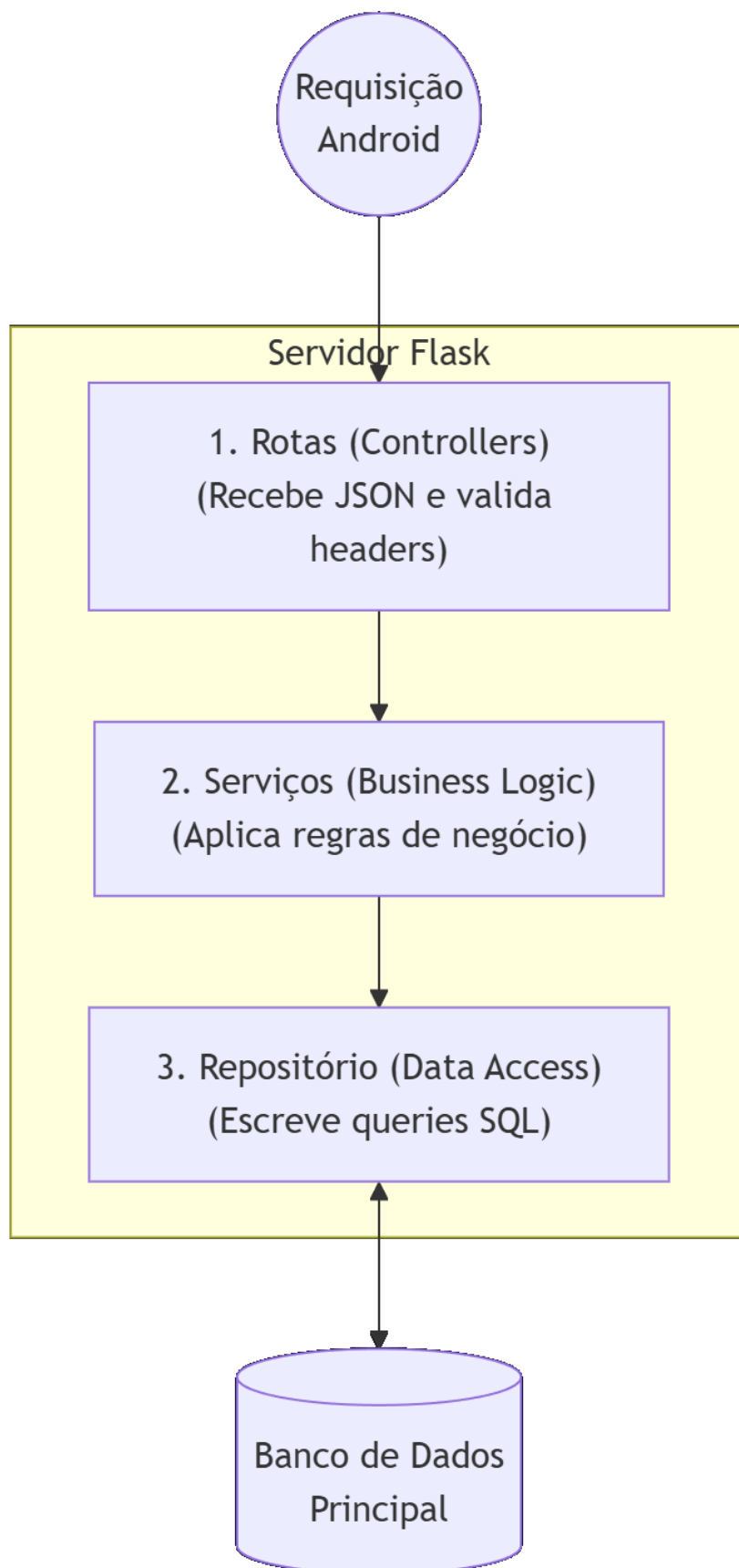


Figura 3: Arquitetura em Camadas do Servidor Flask

7 Considerações de Desempenho e Segurança

Para assegurar o cumprimento dos requisitos não funcionais, a arquitetura impõe as seguintes restrições:

1. **Segurança de Tráfego:** Toda a comunicação entre o Retrofit (Android) e o Flask (Servidor) deve ocorrer obrigatoriamente sob o protocolo HTTPS para evitar interceptação de dados.
2. **Gerenciamento de Bateria:** O serviço de GPS em segundo plano no app do motorista será executado através do componente *Foreground Services* (com notificação persistente), garantindo que o sistema operacional Android não suspenda a captura de dados de forma inesperada.